

3C7 Digital Systems Design

Lab C & Lab D Report

Name: Shipra Singh Student ID: 25370745

LAB C — 6-Bit Ripple Carry Adder

1. Design Description

I implemented a 6-bit ripple carry adder/subtractor in Verilog. It works by chaining six 1-bit full adder modules together (FA0 to FA5). The two inputs x and y are 6-bit numbers in 2's complement format (so they can represent numbers from -32 to +31).

How it handles add vs subtract:

- When sel = 0 (add): y goes straight into the adders, and carry-in to FA0 is 0. Result is $x + y$.
- When sel = 1 (subtract): each bit of y is flipped (XOR with 1), and carry-in to FA0 is set to 1. This converts y to its 2's complement negative, so the result is $x - y$.

Overflow detection: if the carry going INTO the top bit (FA5) is different from the carry coming OUT of FA5, then the result doesn't fit in 6 bits — this means overflow has occurred. The formula is:

```
overflow = carry[5] XOR carry[6]
```

Block Diagram

The diagram below shows how the six full adder stages (FA0-FA5) are chained together. The carry ripples from FA0 through to FA5. sel feeds into both the carry-in of FA0 and into each bit of y (to flip it for subtraction). Overflow is detected at the output as $C5 \text{ XOR } c_out$.

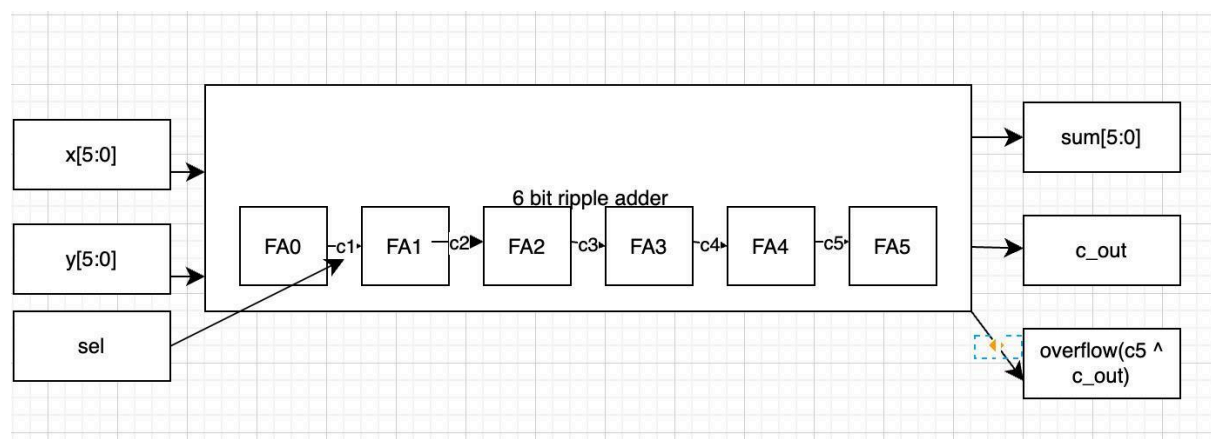


Figure 1 - Block diagram of the 6-bit ripple carry adder/subtractor showing FA0-FA5 chain, carry propagation, and overflow detection

Gate-Level Schematic

The schematic below shows the detailed gate-level design. Each full adder stage (FA0-FA5) receives its XOR'd y input from above (the circle symbols are XOR gates — they flip each y bit when sel=1). The carry ripples left to right from FA0 through to FA5. At the far right, the

final carry (c5) and c_out are XOR'd together to produce the overflow signal. sel also feeds directly into FA0 as the carry-in.

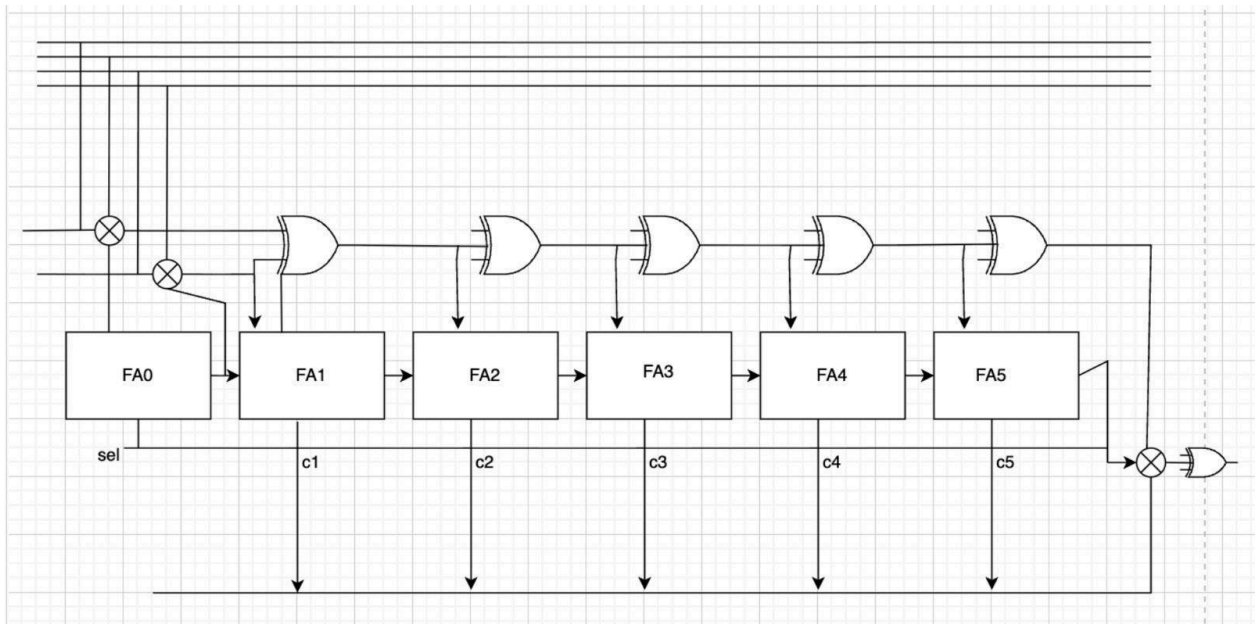


Figure 2 - Gate-level schematic showing XOR pre-processing of y bits, FA0-FA5 carry chain, and overflow XOR at the output

2. Module Ports

Port	Direction	Size	What it does
x	input	6 bits	First number (2's complement)
y	input	6 bits	Second number (2's complement)
sel	input	1 bit	0 = add, 1 = subtract
sum	output	6 bits	Result of the operation
c_out	output	1 bit	Carry out from the top bit
overflow	output	1 bit	1 if the result doesn't fit in 6 bits

3. Verilog Code

full_adder.v

```
module full_adder(A, B, Cin, S, Cout);
    input  A, B, Cin;
    output S, Cout;
    assign S    = A ^ B ^ Cin;
    assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

sixbit_ripple_adder.v

```
module sixbit_ripple_adder(x, y, sel, overflow, c_out, sum);
    input  [5:0] x, y;
    input      sel;
    output     overflow, c_out;
    sum
```

```

output [5:0] sum;
wire [5:0] b_in; // y or flipped-y depending on sel
wire [6:0] carry; // carry[0]=carry-in, carry[6]=carry-out
assign b_in    = y ^ {6{sel}}; // flip y bits when sel=1
assign carry[0] = sel;         // carry-in = 1 for subtract
genvar i;
generate
    for (i = 0; i < 6; i = i + 1) begin : fa_chain
        full_adder fa (.A(x[i]), .B(b_in[i]), .Cin(carry[i]),
                        .S(sum[i]), .Cout(carry[i+1]));
    end
endgenerate
assign c_out    = carry[6];
assign overflow = carry[6] ^ carry[5];
endmodule

```

4. Simulation Waveform

The waveform below was captured from Vivado XSIM. All values are shown in hexadecimal. The simulation runs two phases: sel=0 (addition, vectors T01-T06) and sel=1 (subtraction, vectors T07-T13). The cursor sits at 120.000 ns on test vector T11, which is an overflow case ($-17 - 16 = -33$).

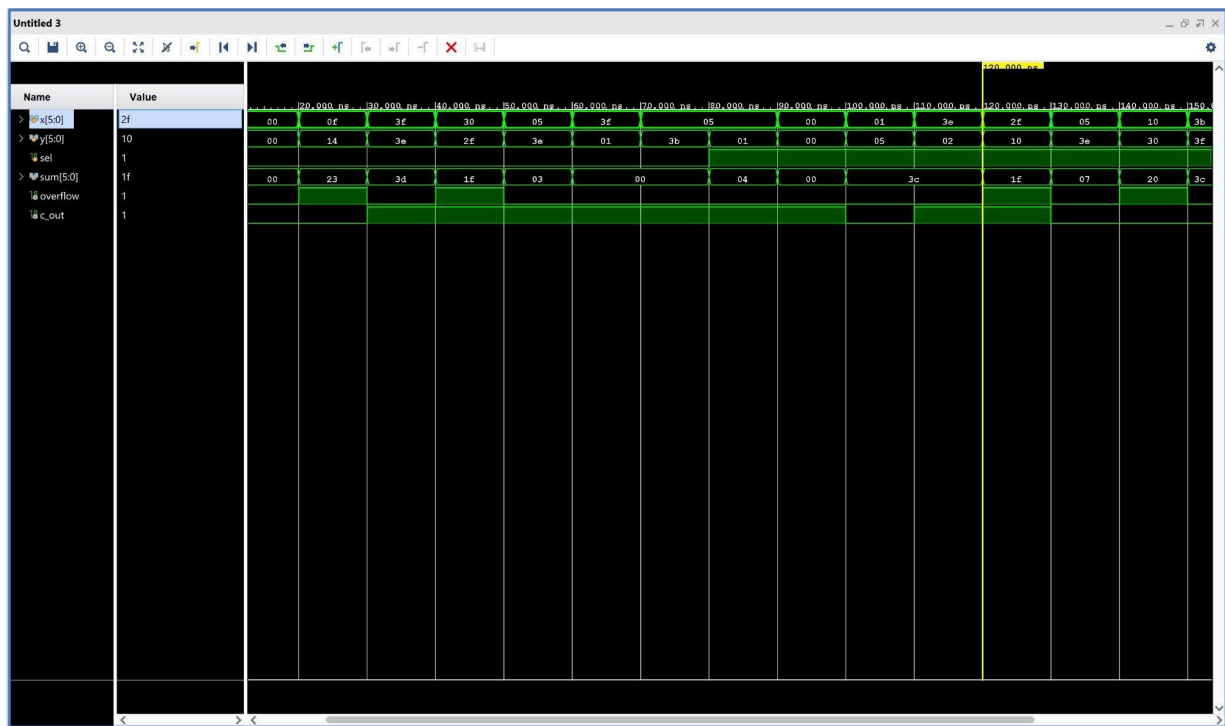


Figure 3 – XSIM simulation waveform. Hex values. Cursor at 120.000 ns => overflow case.

5. Test Vector Results

All 13 values were read from the waveform above. Red rows = overflow detected. The operation column shows what the arithmetic should be.

#	sel	x (hex)	x (dec)	y (hex)	y (dec)	sum (hex)	sum (dec)	c_out	ovfl	Operation
T01	0	00	0	00	0	00	0	0	0	0 + 0 = 0

T02	0	0f	+15	14	+20	23	-29	0	1	+15 + +20 = +35 → OVERFLOW
T03	0	3f	-1	3e	-2	3d	-3	1	0	-1 + (-2) = -3
T04	0	30	-16	2f	-17	1f	+31	1	1	-16 + (-17) = -33 → OVERFLOW
T05	0	05	+5	3e	-2	03	+3	1	0	+5 + (-2) = +3
T06	0	3f	-1	01	+1	00	0	1	0	-1 + +1 = 0
T07	1	05	+5	01	+1	04	+4	1	0	+5 - +1 = +4
T08	1	00	0	00	0	00	0	1	0	0 - 0 = 0
T09	1	01	+1	05	+5	3c	-4	0	0	+1 - +5 = -4
T10	1	3e	-2	02	+2	3c	-4	1	0	-2 - +2 = -4
T11	1	2f	-17	10	+16	1f	+31	1	1	-17 - +16 = -33 → OVERFLOW [cursor 120 ns]
T12	1	05	+5	3e	-2	07	+7	0	0	+5 - (-2) = +7
T13	1	10	+16	30	-16	20	-32	0	1	+16 - (-16) = +32 → OVERFLOW

6. Overflow Examples Explained

T02 — Addition overflow: +15 + +20 = +35

The 6-bit range is -32 to +31. Adding +15 and +20 gives +35 which is too large. The carry going into FA5 is 1, but carry out of FA5 is 0, so overflow = 1 XOR 0 = 1. The sum wraps around to -29 (0x23 in hex).

T04 — Addition overflow: -16 + -17 = -33

Adding two large negative numbers gives -33 which is below -32. Carry into FA5 = 0, carry out = 1, so overflow = 0 XOR 1 = 1. The sum wraps to +31 (0x1f in hex).

T11 — Subtraction overflow: -17 - 16 = -33 [cursor at 120 ns]

-17 minus 16 should give -33, but -32 is the minimum. Overflow is correctly flagged. The sum shows 0x1f (+31) which is the wrapped result. c_out = 1, overflow = 1.

T13 — Subtraction overflow: +16 - (-16) = +32

+16 minus -16 is the same as +16 + +16 = +32, which exceeds +31. The sum wraps to -32 (0x20). c_out = 0, overflow = 1.

7. Test Coverage

The 13 test vectors cover all the important combinations:

- Both inputs zero (T01, T08)
- Both inputs positive — with and without overflow (T02 overflows, T07 doesn't)
- Both inputs negative — with and without overflow (T04 overflows, T03 doesn't)
- Mixed signs, positive and negative (T05, T06, T09, T10, T12)
- Subtraction overflow from both directions (T11 and T13)

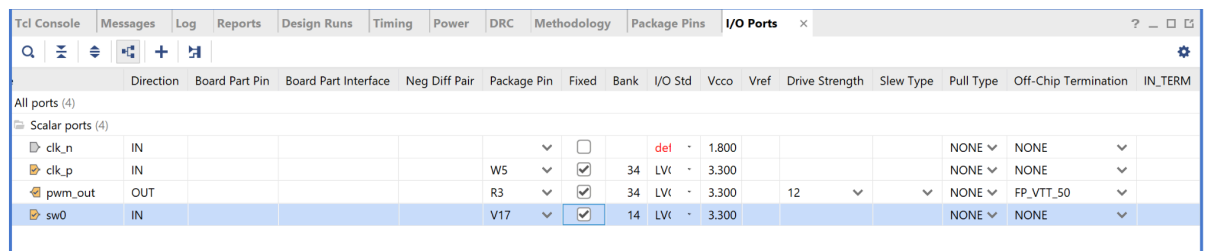
LAB D — Vivado Implementation on BASYS 3

Part 1 — Vivado Tutorial (PWM Modulator)

This part followed the tutorial to implement a PWM (Pulse Width Modulator) design in Vivado. The design outputs a PWM signal shaped like a sine wave. Switch sw0 toggles between two frequencies (1 Hz and 3.5 Hz).

Steps

- Opened Vivado 2025.2 and created a new RTL project called 'modulator', targeting theBasys 3
- Added the VHDL source files from tutorial_code.zip
- Created the constraints file (modulator.xdc) to map the 3 ports to physical pins on the board: pwm_out → pin R3, clk_p → pin W5, sw0 → pin V17.
- Used the Timing Constraints Wizard to set a 100 MHz clock on clk_p.
- Ran synthesis and implementation through the Flow Navigator, both completed without errors
- Opened the Timing Summary and Utilization reports



The screenshot shows the 'I/O Ports' window in Vivado. It displays a table of ports and their physical pin assignments. The ports are: clk_n (IN), clk_p (IN), pwm_out (OUT), and sw0 (IN). The pin assignments are: clk_p to W5, pwm_out to R3, and sw0 to V17. The table also shows the I/O standard (LVCMOS33), voltage (3.300V), and other parameters for each pin.

Port	Direction	Board Part Pin	Board Part Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
clk_n	IN								1.800				NONE	NONE	
clk_p	IN				W5	☒	34	LVCMOS33	3.300				NONE	NONE	
pwm_out	OUT				R3	☒	34	LVCMOS33	3.300	12			NONE	FP_VTT_50	
sw0	IN				V17	☒	14	LVCMOS33	3.300				NONE	NONE	

Timing Report

The timing report confirmed the design meets its 100 MHz timing target. Key numbers:

Metric	Value	What it means
Worst Negative Slack (WNS)	2.902 ns	Design is 2.9 ns faster than required — timing passes
Total Negative Slack (TNS)	0.000 ns	No failing timing paths at all
Worst Hold Slack (WHS)	0.103 ns	Hold timing also passes
Failing endpoints	0	Everything meets timing
Total endpoints	227	227 paths were checked

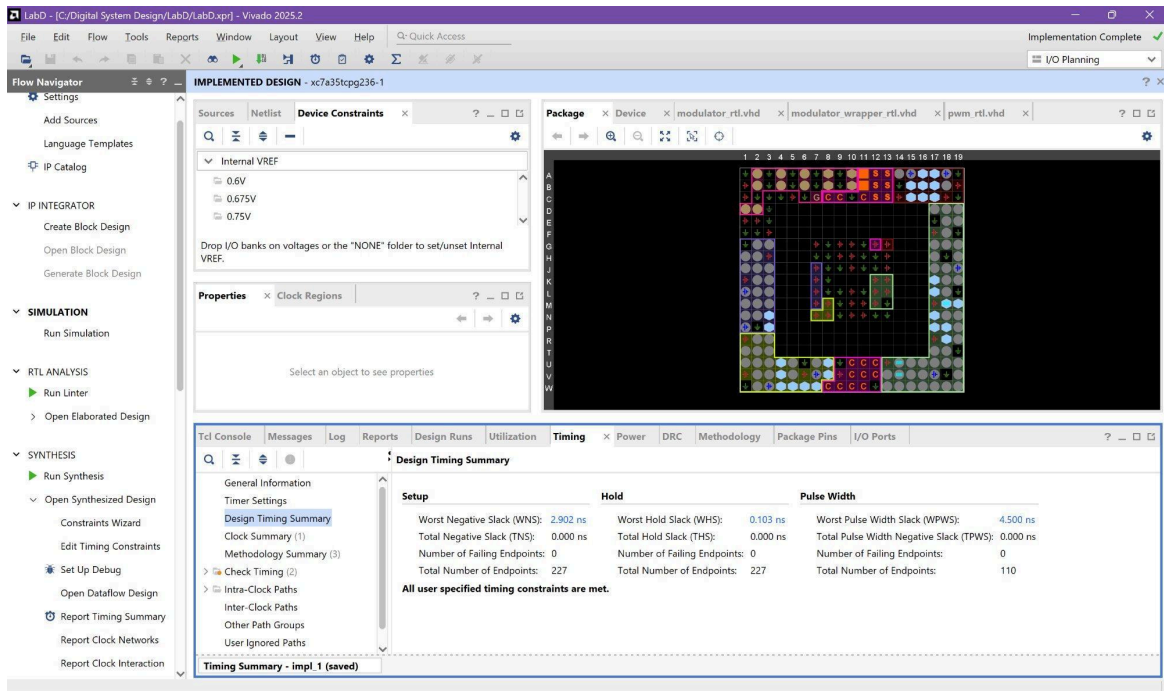


Figure 4 – Design Timing Summary

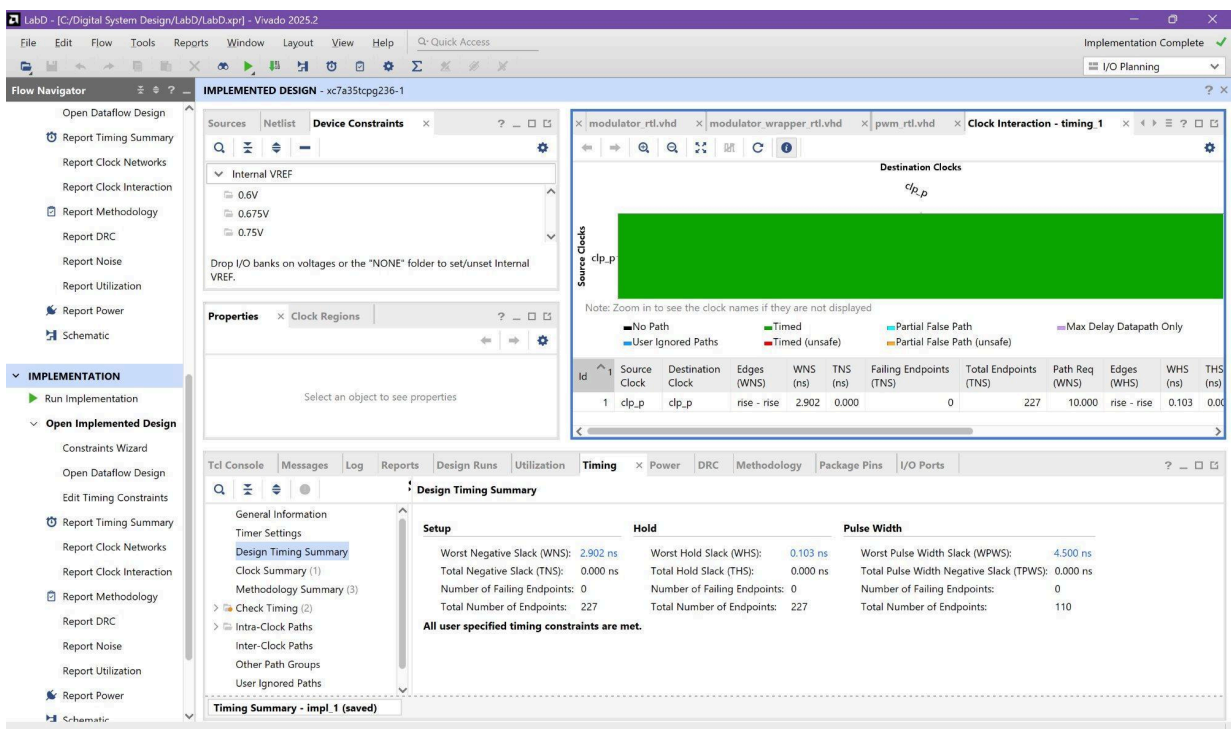


Figure 5 – Clock Interaction report

Power Report

The Power Report showed the total on-chip power consumption of the design after implementation:

Metric	Value	Notes
Total On-Chip Power	0.073 W	Very low — small design
Dynamic Power	0.003 W (4%)	Power used by switching logic
Device Static	0.070 W (96%)	Power just to keep the chip on
Clocks	0.001 W (36%)	Largest dynamic consumer
Junction Temperature	25.4°C	Very cool — no thermal concern
Thermal Margin	59.6°C	Plenty of headroom before overheating

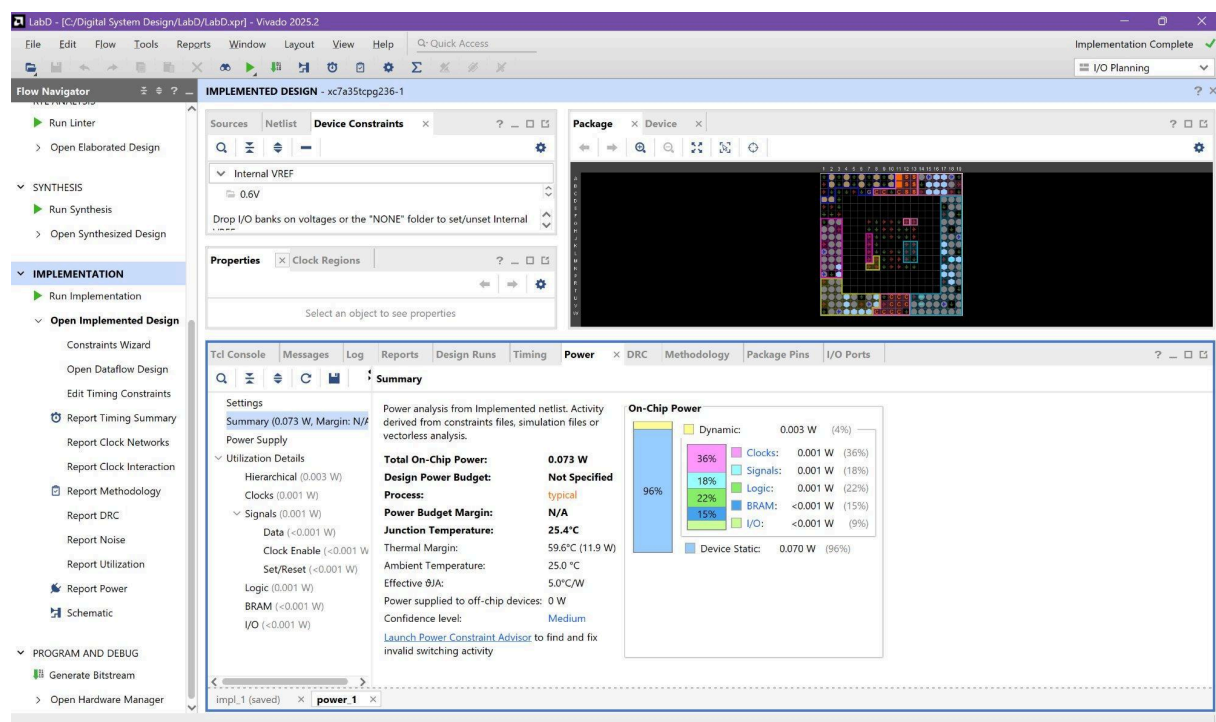
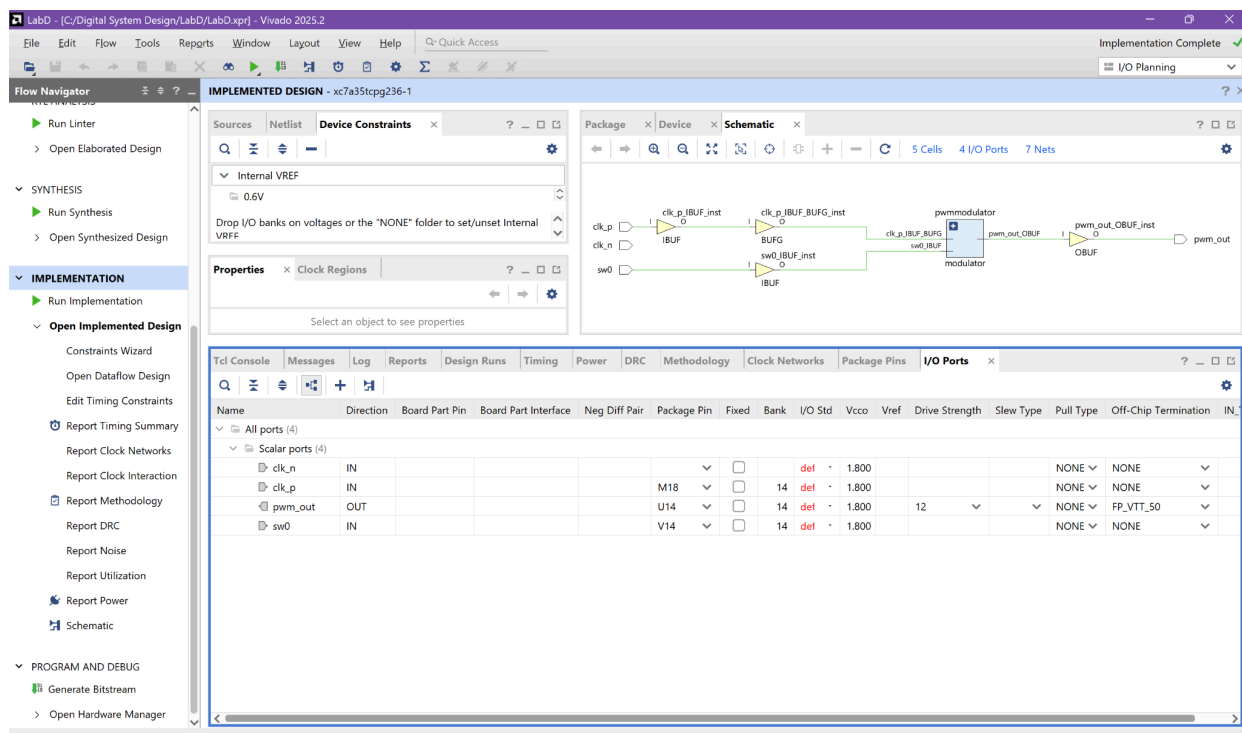
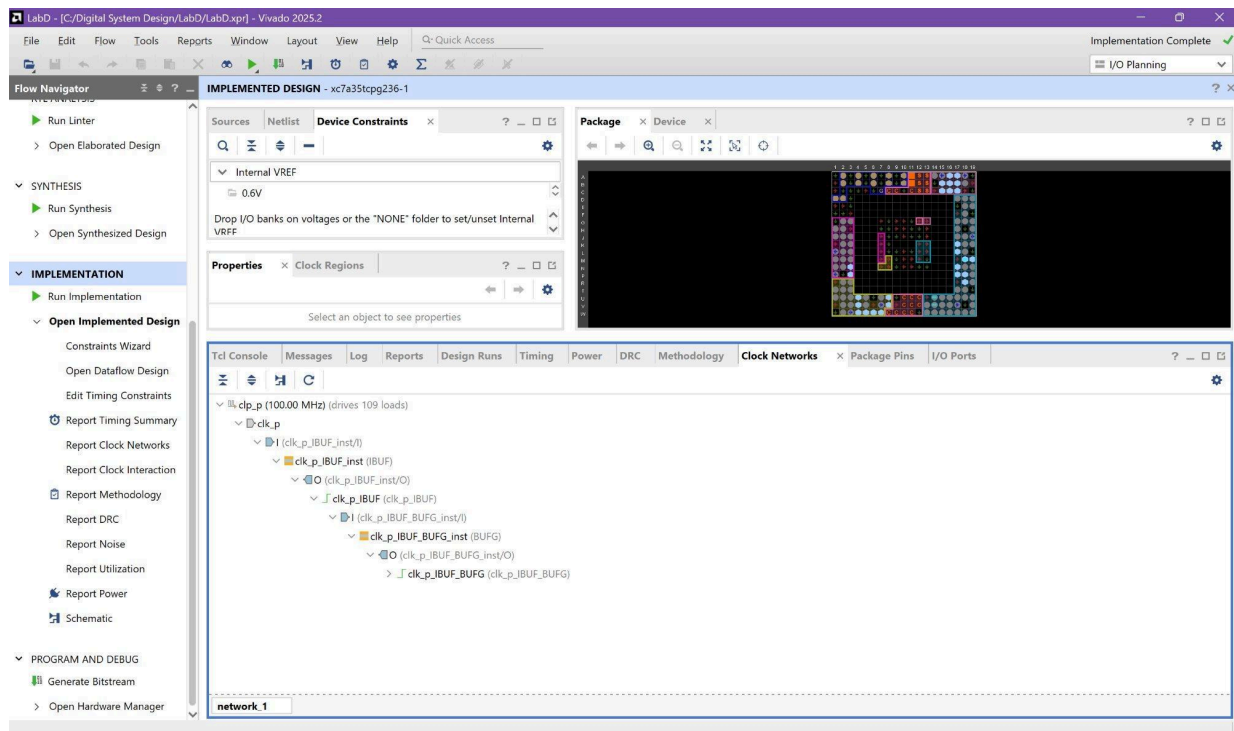


Figure 6 - Power Report

Clock Networks Report

The Clock Networks report shows that a single 100 MHz clock (clk_p) drives 109 loads in the design. The clock goes through an IBUF (input buffer) and then a BUFG (global clock buffer) before reaching all the flip-flops and logic. This is the standard clock path for Xilinx FPGAs.



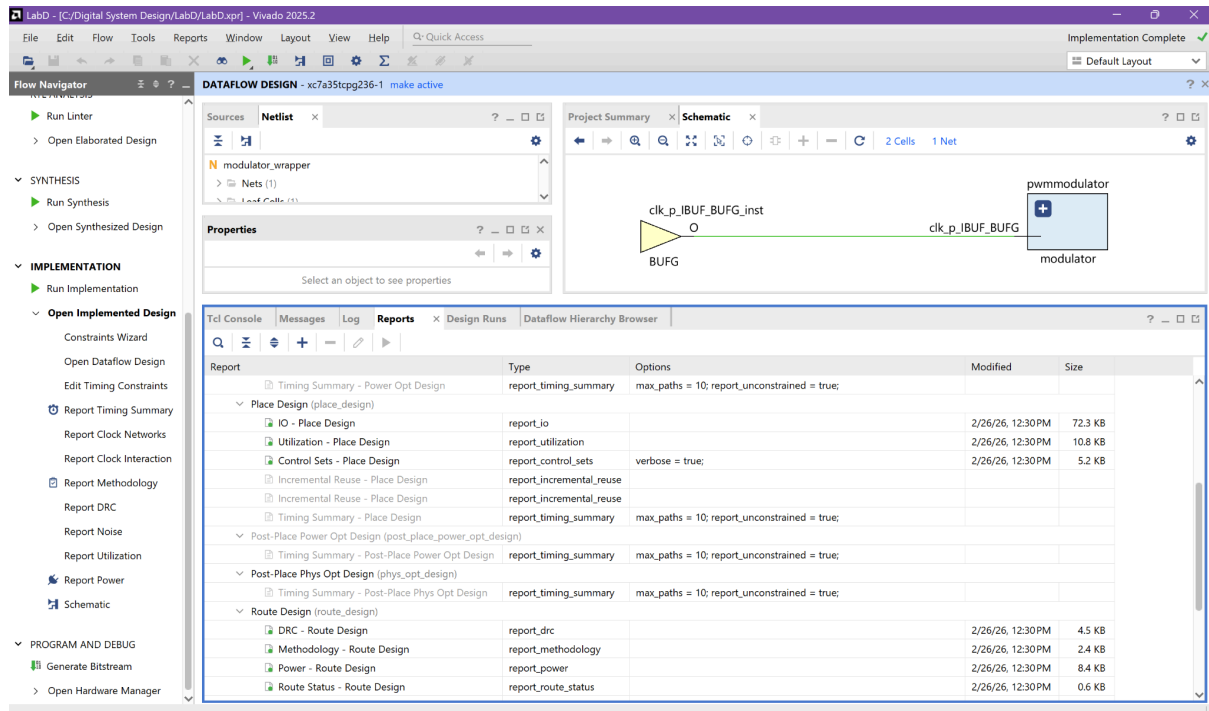


Figure 9 - List of All Reports Generated

Conclusion of Part-1

- Every top-level port must have a pin location AND an I/O standard set in the XDC file or the design won't implement
- The Timing Constraints Wizard makes it easy to set up clock constraints it finds the clocks automatically from the synthesised design
- Positive WNS means timing passes. The bigger the number, the more headroom you have
- The power report shows the design only uses 0.073 W total, 96% of that is just the static power of the chip itself, not the actual logic
- The clock goes through IBUF then BUFG, this is standard practice in Xilinx FPGAs to drive clocks reliably across the whole chip

****Figure 12 – Vivado Utilization Report — Page 12 Along with Part 2**

Part 2 — Bargraph on BASYS 3

I implemented a bargraph on the BASYS 3 board. The design uses 8 of the 16 on-board LEDs to show a level indicator. A clock divider slows the 100 MHz clock down so the pattern is visible.

Source files

File	What it does
bargraphtest.v	Top-level file. Connects everything together and links to the physical pins.
bargraph.v	The actual bargraph logic. Reads the switch value and turns on that many LEDs.
clock.v	Slows the 100 MHz clock down to about 1 Hz so you can see the LEDs change.
bargraphtest.xdc	Constraints file. Maps the clock, switches and LEDs to the correct physical pins on the BASYS 3 board.

Steps

- Created a new Vivado project targeting the BASYS 3 board
- Added bargraphtest.v, bargraph.v, clock.v and the XDC constraints file
- Clicked 'Generate Bitstream', Vivado ran synthesis, implementation and bitstream generation automatically
- Connected the BASYS 3 board via USB and opened Hardware Manager
- Clicked Auto Connect, the board was detected
- Programmed the FPGA, the LEDs started showing the bargraph pattern
- Set my student number in binary on the slide switches and took a photo

After programming, the 8 LEDs displayed a VU-meter style pattern cycling at approximately 1 Hz (consistent with the clock.v divider). The slide switches were set to my student number in binary and photos were taken of the board showing the LED pattern:

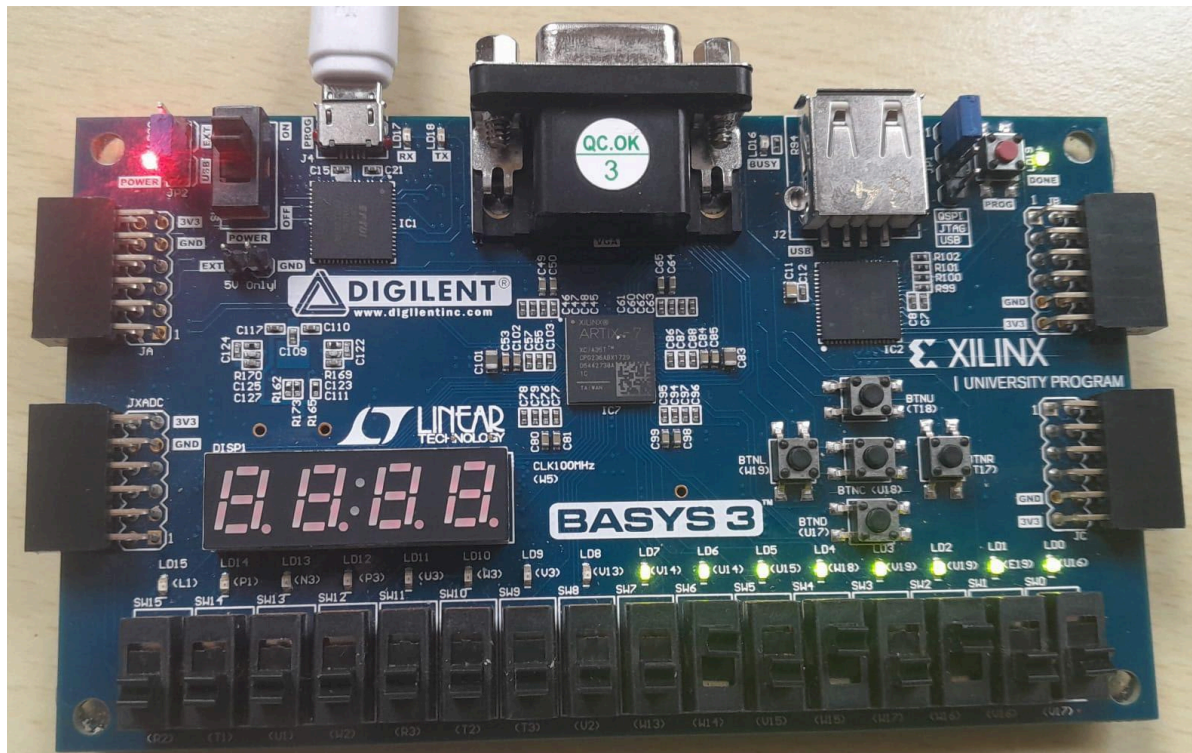


Figure 10 - BASYS 3 board programmed and running the bargraph design. Green LEDs visible, 7-segment display active.

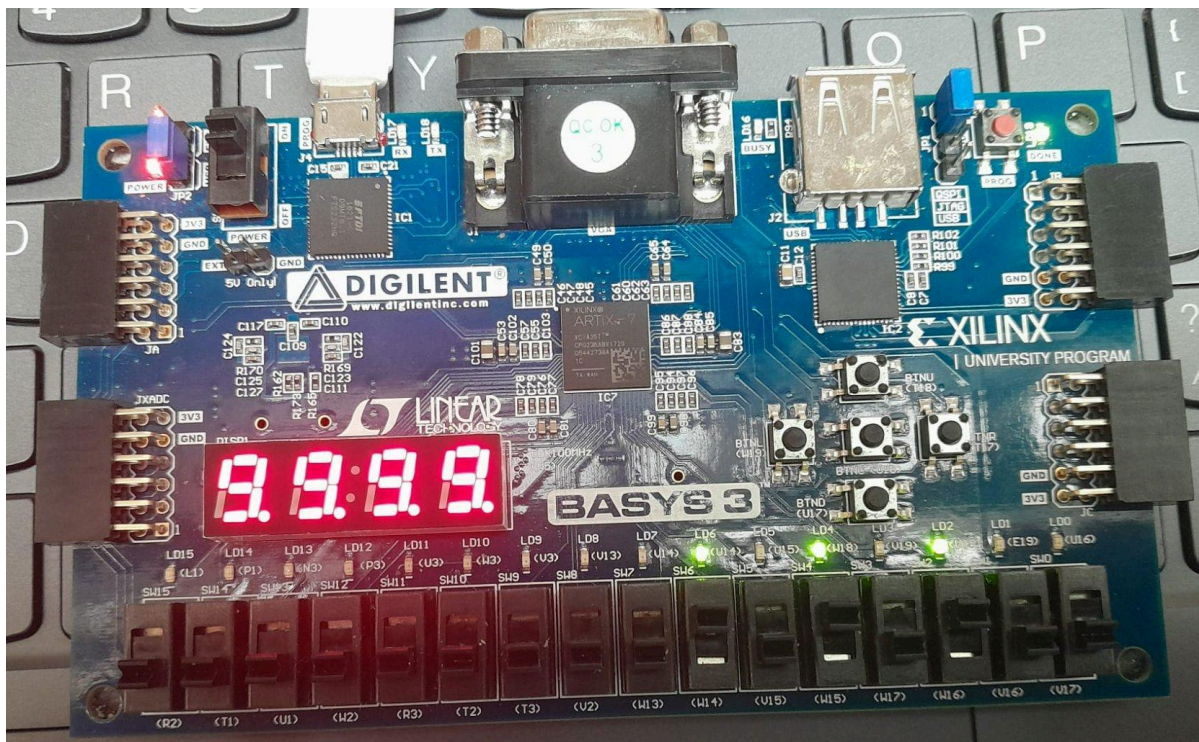


Figure 11 - BASYS 3 board showing LED bargraph pattern with student number set on slide switches.

Utilization Report

These numbers were read from the Vivado Utilization Report after implementation on the BASYS 3:

Resource	Used	Available	Util %	Notes
LUT	109	20,800	0.52%	Logic for the bargraph and clock divider
FF	108	41,600	0.26%	Registers in the clock divider counter
BRAM	0.50	50	1.00%	Used for clock divider (inferred by Vivado)
IO	3	106	2.83%	Clock input, switches and LED outputs

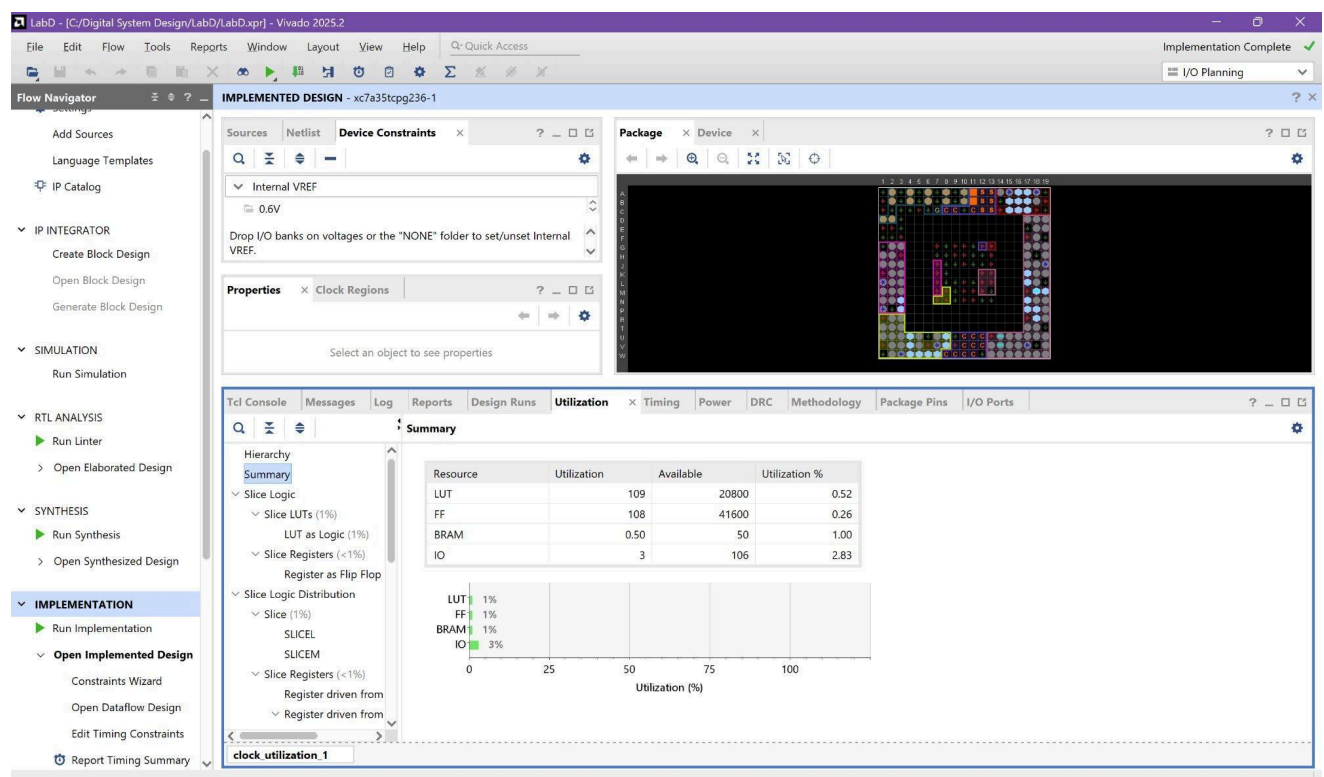


Figure 12 – Vivado Utilization Report

Conclusion of Part-2

- After programming, the 8 LEDs lit up and cycled through a bargraph pattern at about 1 Hz
- The pattern stepped up and down smoothly, behaving like a VU meter
- When I changed the slide switches, the number of LEDs lit changed accordingly
- The board stayed programmed until it was powered off